

Memory Under Pressure

Supporting explainer for the DataForge 2026 Pathway-track submission on Associative Memory and Fast Weights. All screenshots below are of the live artifact; every number in them was computed in the browser at capture time. Interactive artifact and source: github.com/jainraunak944/dataforge, branch `claude/memory-under-pressure-p6vp89`.

The claim the artifact teaches: for the additive memory in this explainer, storing two associations with orthogonal unit keys permits exact recall; increasing their key overlap introduces calculable interference while the recurrent matrix keeps the same shape and no trained parameters change.

Why a four-number memory is worth an essay

A Transformer remembers by keeping every token: its key–value cache grows with context, and attention re-scans it at each step. An old alternative is back at the frontier: store relationships in *rapidly changing connections* — fast weights — and keep the state a fixed size. The minimal version, with column vectors, is a single matrix updated by an outer product per association and read by one matrix–vector product:

$$M_t = \lambda \cdot M_{t-1} + v_t k_t^T \quad \text{retrieved}(q) = M_t q$$

This is precisely the memory a linear-attention layer maintains — Schlag, Irie and Schmidhuber proved the equivalence in 2021 [3]. What makes it teachable is that at size 2×2 *nothing is hidden*: the learner can watch every scalar as memory succeeds, degrades, and fails.

Two writes into one 2x2 matrix LIVE COMPUTATION

The pairs to store — cue → value

● A

1.000
0.000
k_A
1.000
0.000
v_A

▲ B

0.600
0.800
k_B
0.000
1.000
v_B

Each write adds an outer product

$$M_t = \lambda M_{t-1} + v_t k_t^\top$$

0.000	0.000
0.600	0.800

write 2: ▲ B $v_B k_B^\top$

Memory after step 2

1.000	0.000
0.600	0.800
M_2	

◀ Back

Step ▶

Play

Writing is **addition**: each pair drops a rank-1 stamp $v k^\top$ onto the same four numbers.

Nothing is stored “in a slot”; both associations share every cell. With the current

overlap ($\rho = 0.60$), the matrix ends at $M_2 = \begin{bmatrix} 1.00 & 0.00 \\ 0.60 & 0.80 \end{bmatrix}$.

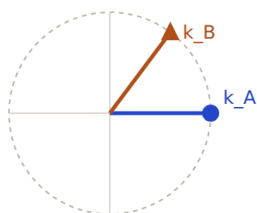
Step 1 — two associations written into four shared numbers. The stepping animation replays real computed matrix states (Play/Pause/Step, reduced-motion aware).

One slider, one lesson

Everything interesting is controlled by the key overlap $\rho = k_A \cdot k_B$. The artifact opens already running at $\rho = 0.6$; the learner drags one slider and watches truth sit beside estimate. Reading with cue A returns $[1, \rho]$ against a desired $[1, 0]$ — the other association leaks in, scaled by exactly ρ , and the displayed L2 error equals ρ . At $\rho = 0$ recall is exact. At $\rho = 1$ both cues are identical, both reads return $[1, 1]$, and the artifact reports a *tie*: the cue carries no information that could distinguish the two memories, and declaring a winner would be fabrication.

One slider: key overlap $\rho = k_A \cdot k_B$ LIVE COMPUTATION

Key overlap ρ (0 = orthogonal cues, 1 = identical cues)



Both keys stay unit length; the slider only rotates $\blacktriangle k_B$ toward $\bullet k_A$.

Query with cue $\bullet A$

1.000

0.000

desired (ground truth)

1.000

0.600

retrieved = $M \cdot \text{cue}$

1.000

0.600

reference $\sum v_i(k_i \cdot q)$

L2 error 0.600 recurrent vs

reference agree to 0.0e+0

Query with cue $\blacktriangle B$

0.000

1.000

desired (ground truth)

0.600

1.000

retrieved = $M \cdot \text{cue}$

0.600

1.000

reference $\sum v_i(k_i \cdot q)$

L2 error 0.600 recurrent vs

reference agree to 0.0e+0

The closed forms are visible in the numbers: reading with k_A returns $[1, \rho]^T$, reading with k_B returns $[\rho, 1]^T$. At $\rho = 0$ recall is exact. As overlap grows, each cue drags in ρ of the other value — and the L2 error is exactly ρ . Drag the slider and watch the prediction hold.

The **reference** column is an independent computation that keeps every (key, value) pair in a list and evaluates $\sum_i v_i(k_i \cdot q)$ directly — the same algebra with a different storage strategy. Agreement between the two columns is a property of this *linear* operation, not of softmax attention. The stated tolerance is 10^{-9} .

Step 2 — truth beside estimate. The third column is an independently coded explicit-history reference ($\sum \lambda^{t-i} v_i(k_i \cdot q)$); the recurrent and reference paths must agree within a stated 10^{-9} tolerance, so the app cannot confirm itself circularly.

The interference is not mysterious, and the artifact makes the learner point at its source: expanding the read gives $M k_A = v_A(k_A \cdot k_A) + v_B(k_B \cdot k_A)$. The second term — value B times the key overlap — lives in identifiable matrix cells, and clicking any cell decomposes it into what each write contributed.

Point at the interference LIVE COMPUTATION

Query ● **A** should return $[1, 0]^\top$ but returns $[1, 0.60]^\top$. Algebraically the read expands to

$$Mk_A = v_A (k_A \cdot k_A) + v_B \underbrace{(k_B \cdot k_A)}_{\rho} = v_A + \rho v_B$$

The extra signal is the **cross term** ρv_B — association B leaking through, scaled by exactly the key overlap. Click the matrix cells to see which write put what where. Because the cue is $[1, 0]^\top$, the read touches only column 1; the highlighted cell is the one that leaks.

1.000	0.000
0.600	0.800

M — click a cell ↵

Cell M[2][1] = 0.600

WRITE	CONTRIBUTION
● A $v_A k_A^\top$	0.000
▲ B $v_B k_B^\top$	0.600
sum (what M stores)	0.600

This is it: B wrote $\rho = 0.60$ here, and the cue k_A reads this cell at full strength. This single number is the interference you saw above.

Step 3 — the cross term, locatable. With cue $[1, 0]$, only column 1 is read; the highlighted cell (written by B, value ρ) is the entire interference.

Honesty rules baked into the build. The memory engine never receives item identities or ground truth — only vectors; the evaluator holds the truth and computes errors. Raw readouts are labeled scores, never probabilities. All caps (history ≤ 64 pairs, $\lambda \in [0, 1]$, $|\text{components}| \leq 4$) are stated in the interface. 31 unit tests check hand-calculated fixtures; 21 browser checks include driving the slider and verifying the rendered numbers match the closed form — a machine check that no displayed value is painted.

Where this lives in BDH — and in BDH-CQ

The Dragon Hatchling (BDH) reformulates attention as synaptic memory [4]. Its GPU formulation carries a per-layer fast state p updated, per token, by an outer product of a learned value with the current sparse activation, then decayed (Eq. 8 of the paper): $p_{t, \ell} := (p_{t-1, \ell} + \text{LN}(E y_{t, \ell-1}) x_{t, \ell}^\top) \cup$, read as px inside ReLU-gated machinery. The mapping to the toy is one-to-one — write, read, decay, state — with the differences stated: the value is learned ($\text{LN}(E y)$), the key and query are the *same* non-negative activation x , and everything runs at n up to $\sim 10^6$ dimensions with $\sim 5\%$ sparsity (author-reported). The artifact's variable-role table answers, for every symbol, the question the toy trains: *does it change during inference?* The state does; the trained matrices E , D_x , D_y do not. BDH is not a Mamba-style state-space model, and the artifact never presents the toy as an official BDH implementation.

The same write and read inside BDH-GPU SOURCE-DERIVED ILLUSTRATION

Which system, precisely: the Dragon Hatchling paper gives two coupled formulations. *Graph BDH* describes neurons and synapses with local rules — attention becomes Hebbian synaptic memory on a graph (its state update is Eq. 6 of the paper). *BDH-GPU* is the tensor-friendly formulation actually trained on hardware (§3.2). It is *not* a Mamba-style state-space model: the paper positions it as linear attention operating in one very high-dimensional, non-negative, sparsely active neuronal space. The concept from our toy shows up in BDH-GPU's layer state update, Eq. (8) of arXiv:2509.26507:

$$\rho_{t,\ell} := (\rho_{t-1,\ell} + \text{LN}(E y_{t,\ell-1}) x_{t,\ell}^\top) U$$

the write — compare the toy's $M_t = \lambda M_{t-1} + v_t k_t^\top$

$$y_{t,\ell} := (D_y \text{LN}(\rho_{t-1,\ell} x_{t,\ell}))^+ \odot x_{t,\ell}$$

the read — inside it sits the toy's Mq (here ρx), wrapped in machinery the toy omits

Every symbol, and whether it changes at inference

SYMBOL	SPACE (PAPER §3.2, §6.1)	ROLE — IN THE TOY'S VOCABULARY	CHANGES DURING INFERENCE?
$x_{t,\ell}$	$(\mathbb{R}^+)^n$, n up to $\sim 10^6$, $\sim 5\%$ active	the key and the query at once — BDH uses the same sparse activation vector for both, where the toy had separate k and q	yes — activation
$y_{t,\ell}$	$(\mathbb{R}^+)^n$	output activation; its low-rank projection makes the next value	yes — activation
$\text{LN}(E y_{t,\ell-1})$	$\mathbb{R}^d$, d = 256 in the paper's runs	the value v_t — computed from activity, not looked up	yes — derived from activations
$\rho_{t,\ell}$	d×n as composed in Eq. (8) — see note below	the memory matrix M — fast “synaptic” state, zeroed at sequence start, written once per token per layer	yes — this is the fast weight state
U	linear map on the time axis	the decay/rotation — the toy's λ generalized (“local rotation or damping... such as ALiBi or RoPE”, Def. 4)	no — fixed convention
E, D_x, D_y	$E \in \mathbb{R}^{d \times n}$, $D_x, D_y \in \mathbb{R}^{n \times d}$ (≈ 3 nd params total)	trained weights — encoder/decoders deciding how to write and read; the toy has no counterpart at all	no — frozen after training

Dimension check (do it yourself): the write $\text{LN}(E y) x^\top$ is $(d \times 1)(1 \times n) = d \times n$, a rank-1 outer product exactly like the toy's $v k^\top$; the read ρx is $(d \times n)(n \times 1) = d$, a matrix–vector product exactly like Mq . Indexing is causal: the read at token t uses ρ_{t-1} — the state *before* this token's write, so no token reads its own write.

One honesty note from our source check: the paper's Fig. 3 caption declares $\rho \in \mathbb{R}^{n \times d}$ while its equations compose ρ as $d \times n$; we follow the composition that makes Eq. (8)'s algebra type-check, and record the discrepancy in docs/RESEARCH.md rather than silently mixing conventions.

What the toy leaves out — deliberately

- **Non-negativity and sparsity:** BDH activations live in $(\mathbb{R}^+)^n$ with ReLU thresholding; ~5% of neurons are active (Empirical Finding 1, §4.1 — author-reported). The toy's vectors go negative and dense.
- **Trained encoder/decoders** E, D_x, D_y that learn *what* to write; the toy hand-picks its vectors.
- **Layer structure and gating:** the read is thresholded and gated ($\odot x$), and layers stack with a shared state pattern; the toy is one bare matrix.
- **The graph view:** per-synapse local rules, excitatory/inhibitory circuits (Eq. 6, Table 1) — with reported (author-evaluated) consequences: monosemantic synapses (§6.3) and heavy-tailed connectivity (Fig. 11).
- **Scale:** BDH-GPU runs at n up to $\sim 10^6$ with GPT-2-class language performance at 10M–1B parameters (Fig. 7, Tables 4–5 — author-reported, Europarl). The toy is 2×2 on purpose: every scalar visible.

This page's simulation is an **original toy computation** of the shared mechanism. It is not an official BDH model, does not use BDH code or checkpoints, and demonstrates nothing about trained monosemantic synapses. Adding linear attention to a Transformer does not recreate BDH — the paper derives the architecture from local neuron–synapse dynamics, of which the fast-weight write is one ingredient.

Check yourself: during BDH inference, which quantity changes?

The synaptic state ρ (equivalently σ) — and the activations x, y

The trained matrices E, D_x, D_y

Both the state and the trained matrices

And BDH-CQ — the same split, one level up SOURCE-DERIVED ILLUSTRATION

BDH-CQ (arXiv:2608.09888, 2026) applies this vocabulary to learning *from demonstrations*. Reading demonstrations $D_1 \dots D_K$ builds a contextual memory by a recurrent update (its Eq. 1, §3.2):

$$S_t = U_\theta(S_{t-1}, D_t)$$

with θ — the trained parameters — held fixed. The report explicitly relates this to “attention, fast-weight memory, and linear-attention views of contextual association,” with linear attention as the special case where the state accumulates additively per demonstration, $S_t = S_{t-1} + U_\theta(D_t)$ — precisely our toy's write, one demonstration per stamp. Answering then iterates a latent workspace against the frozen memory (Eqs. 2–4): $H_{r+1} = F_\theta(H_r, S_K)$ — recurrent latent reasoning, distinct from the memory write. The report states plainly: “Neither task identifiers nor evaluation-task demonstration pairs participate in training, and no parameters are updated at inference

time” (§1). Adaptation lives entirely in state — the toy’s lesson, load-bearing.

Evidence discipline: BDH-CQ reports 118/400 = 29.50% pass@2 on ARC-AGI-1 at ≈ 0.85 H200 GPU-seconds ($\approx \$0.0007$) per task, with accuracy scaling across LOW/MEDIUM/HIGH latent-effort settings (21% / 27% / 29.5%; Tables 1 and 5) — **author-reported**, audited black-box by co-authors, and the report itself notes its update rules and dimensions are proprietary. Our toy demonstrates the memory *mechanism family*; it reproduces nothing about ARC or the full reasoning system.

Step 5 — the BDH module: sourced equations (with equation numbers), a variable-role table with dimensions, the machinery the toy omits, and a self-check question. Below it, the BDH-CQ section separates state updates from parameter updates at the system level.

BDH-CQ [5] lifts the same distinction to task acquisition: reading demonstrations accumulates a contextual memory $S_t = U_\theta(S_{t-1}, D_t)$ — the report explicitly names additive per-demonstration accumulation as its linear-attention special case — and answering iterates a latent workspace against that frozen memory, with “no parameters ... updated at inference time” (§1). On ARC-AGI-1 it reports 118/400 (29.50%) pass@2 at roughly 0.85 H200 GPU-seconds per task, with accuracy scaling across latent-effort levels (21% $\rightarrow$ 27% $\rightarrow$ 29.5%). Those numbers are author-reported; the audit that reproduced them was run by co-authors (semi-independent), the update-rule internals are proprietary, and the report itself flags a training-data overlap caveat — all of which the artifact states next to the numbers.

Breaking it on purpose

The sandbox hands over decay λ and repeated writes, with presets for the claim’s edge cases — orthogonal, partial overlap, identical cues, repeated writes, decay. Repeating writes scales magnitudes while the matrix stays 2×2 ; decay fades old writes geometrically (at $\lambda = 0$ only the newest survives, weight $\lambda^0 = 1$ by stated convention). Neither removes interference — which is why DeltaNet replaces the blind add with an error-correcting write (its motivation quotes key “collisions” directly [1]) and Gated DeltaNet adds learned erasure [2]. At scale the stakes are stark: with unlimited context on WikiText-103, the purely additive model degrades beyond perplexity 260 while the delta-rule variant of the same size reaches 29.4 [3, Table 4].

LIVE COMPUTATION

Orthogonal cues

Partial overlap

Identical cues

Repeated writes

Decay

Orthogonal cues: $\rho = 0$ — the claim predicts exact recall of both associations.

Partial overlap: $\rho = 0.6$ — the claim predicts interference of exactly 0.6 in the off component.

Identical cues: $\rho = 1$ — both reads return [1, 1]; the cue cannot distinguish A from B.

Repeated writes: Write A and B three times each ($\lambda = 1$): magnitudes triple, shape does not change.

Decay: $\lambda = 0.5$ — earlier writes fade; the newest write keeps full weight.

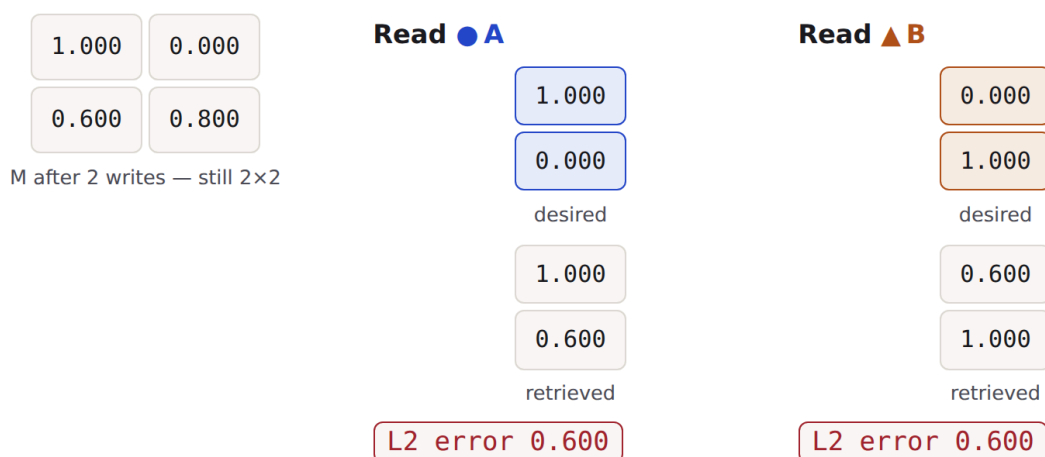
Key overlap ρ



Decay λ — the guided lesson fixed $\lambda = 1$; here it is yours



Writes of each pair (A,B,A,B...) — capped at 32 each so the retained-pairs comparison below stays bounded at 64 pairs



Honest memory accounting

STORAGE STRATEGY	SCALAR SLOTS (FLOAT64)	BYTES	GROWS WITH WRITES?
Recurrent matrix M ($d_k \cdot d_v$)	4	32	No — constant shape (that is the point)
Retained pairs ($N \cdot (d_k + d_v)$), the reference path	8	64	Yes — linear in N (capped at 64 pairs here)

These counts are analytical, for the model state alone under float64. The teaching page additionally holds a bounded write history for the step animation, the fixture and the ground truth. A constant-size *model state* does not mean constant total browser memory, fixed total compute, unbounded lossless storage — or zero interference; the error column above is the price. Not measured GPU RAM; not a Transformer benchmark.

What repeated writes show: with $\lambda = 1$ the readout magnitudes scale with the write count (write everything twice, read twice as much) while the matrix stays 2×2 . What decay shows: with $\lambda < 1$ older writes fade geometrically — at $\lambda = 0$ only the newest write survives (the newest keeps weight $\lambda^0 = 1$ by convention). Neither escape hatch removes interference; delta-rule memories (see the comparison below) attack it differently.

Step 6 — sandbox with honest accounting: 4 scalars of model state versus $N \cdot (d_k + d_v)$ for retained pairs, under a stated dtype — analytical counts, not GPU measurements.

Try it, test it, refute it

The artifact ends by asking the learner to predict a held-out setting before the app computes it, then to say which state changed and what remains a limitation — deterministic feedback, no grader model. Everything is reproducible from the repository: `npm ci && npm test && npm run build && npm run smoke`. The claim-to-source ledger (`docs/RESEARCH.md`) gives an equation-level locator and an evidence label for every scientific statement above — including which results are author-reported and which have no independent reproduction yet.

References

- [1] S. Yang, B. Wang, Y. Zhang, Y. Shen, Y. Kim. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*. NeurIPS 2024. arXiv:2406.06484.
- [2] S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule*. ICLR 2025. arXiv:2412.06464.
- [3] I. Schlag, K. Irie, J. Schmidhuber. *Linear Transformers Are Secretly Fast Weight Programmers*. ICML 2021. arXiv:2102.11174 (background; predates the 2022–2026 window).
- [4] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz. *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. 2025. arXiv:2509.26507.
- [5] B. Engdahl, A. Kosowski, J. Chorowski, et al. *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. 2026. arXiv:2608.09888.
6. Official (toy) BDH implementation, MIT: github.com/pathwaycom/bdh. Pathway explainers: “The Equations of Reasoning”; “Why BDH uses a brain-inspired architecture”.